

# 程序运行说明

班级：待填写

产品主题：Vimo Desktop 视频理解与问答应用

小组长：待填写

本说明用于在 Windows 环境中从源代码启动 Vimo Desktop。程序由 Electron 桌面端和 Python VideoRAG 服务组成，开发模式下必须先启动后端，再启动桌面端。

## 1 程序结构说明

Vimo Desktop 按调用链分为页面层、桌面通信层、Python API 层和 VideoRAG 算法层。用户在页面中导入视频和提问，Electron 主进程通过 IPC 与本地 HTTP 接口转发请求，Python 后端负责异步任务与状态管理，算法层完成视频分段、特征提取、索引和检索问答。

| 目录或文件                  | 作用                                                                     |
|------------------------|------------------------------------------------------------------------|
| src/renderer           | React 页面、初始化向导、视频导入、会话和问答界面                                            |
| src/preload 与 src/main | 通过 contextBridge 暴露受控 API，并由 Electron 主进程完成 IPC、配置读写和 Python API 转发    |
| python_backend         | videorag_api.py 提供 Flask API 和任务管理；<br>videorag 子目录实现视频处理、多模态索引和检索生成算法 |



## 2 程序运行方法

### 2.1 运行环境

| 项目      | 建议版本或要求                  | 说明                                      |
|---------|--------------------------|-----------------------------------------|
| 操作系统    | Windows 10 或 11<br>64 位  | 本说明采用 PowerShell; macOS 使用相同命令但路径分隔符为 / |
| Python  | 3.11                     | 建议使用 Conda 创建名为 vimo 的独立环境              |
| Node.js | > = 20                   | package.json 的最低版本要求                    |
| pnpm    | 9.10.0                   | 项目锁文件和 packageManager 指定的包管理器           |
| Git     | 可用版本                     | 安装 pytorchvideo 和 ImageBind 源码依赖时使用     |
| 网络与磁盘   | 稳定网络, 建议至少<br>10 GB 空闲空间 | ImageBind 模型约 4.5 GB, 依赖和分析缓存还会占用空间     |
| FFmpeg  | 建议安装并加入<br>PATH          | MoviePy 处理视频和音频时需要                      |

CPU 可以运行程序, 但视频分析和 ImageBind 加载会较慢; 如使用 NVIDIA

GPU, 应安装与本机 CUDA 环境匹配的 PyTorch。

## 2.2 创建 Python 环境

在 PowerShell 中执行以下命令。首次安装需要联网，Git 依赖下载时间较长。

```
conda create --name vimo python=3.11 -y
conda activate vimo
python -m pip install --upgrade pip

python -m pip install numpy==1.26.4 torch==2.1.2 torchvision==0.16.2
torchaudio==2.1.2
python -m pip install moviepy==1.0.3
python -m pip install git+https://github.com/Re-
bin/pytorchvideo.git@58f50da4e4b7bf0b17b1211dc6b283ba42e522df
python -m pip install --no-deps
git+https://github.com/facebookresearch/ImageBind.git@3fcf5c9039de97f6ff552
8ee4a9dce903c5979b3

python -m pip install timm ftfy regex einops fvcore eva-decord==0.6.1
iopath matplotlib types-regex cartopy
python -m pip install neo4j hnswlib xxhash nano-vectordb
python -m pip install tiktoken openai tenacity dashscope
python -m pip install flask psutil flask_cors setproctitle pywin32
```

## 2.3 安装桌面端依赖

进入 Vimo-desktop 根目录，使用项目指定的 pnpm 安装依赖。

```
cd <项目目录>\Vimo-desktop
corepack enable
corepack prepare pnpm@9.10.0 --activate
pnpm install
```

若当前 Node.js 未提供 Corepack，可执行 `npm.cmd install -g pnpm@9.10.0` 后再运行 `pnpm install`。

## 2.4 启动 Python 后端

打开第一个 PowerShell 窗口并保持运行：

```
conda activate vimo  
cd <项目目录>\Vimo-desktop\python_backend  
python videorag_api.py
```

服务默认使用 64451 端口；该端口被占用时，会在 64451 至 64470 范围内选择可用端口。出现 VideoRAG API is running 或 Flask listening 信息后再启动桌面端。

## 2.5 启动桌面端

打开第二个 PowerShell 窗口：

```
cd <项目目录>\Vimo-desktop  
pnpm dev
```

开发模式下 Electron 会扫描已启动的后端服务。若后端未运行，界面会持续等待或提示未找到服务。

## 2.6 首次启动配置

- 1. 选择存储目录** 选择有写权限且空间充足的目录，用于保存模型、配置、会话和索引数据。
- 2. 准备 ImageBind** 点击 Start Download 下载约 4.5 GB 的模型，等待状态变为 Completed。模型文件保存在 <存储目录>\imagebind\_huge\imagebind\_huge.pth。
- 3. 配置 OpenAI** 填写 Base URL、OpenAI API Key、Processing Model 和 Analysis Model。默认模型名为 gpt-4o-mini。

**4. 配置 DashScope** 填写 DashScope API Key。默认视频描述模型为 qwen-vl-plus-latest, 语音识别模型为 paraformer-realtime-v2。

**5. 完成初始化** 保存配置后进入主界面。当前后端实际运行视频分析时要求 OpenAI 和 DashScope 两项 API Key 均已配置。

程序没有内置登录账户或密码。API Key 必须由使用者从对应服务商获取, 文档和截图中不要记录真实密钥。存储目录记录在 %USERPROFILE%\videorag-bootstrap.json, 其他设置保存在所选目录的 config.json。

## 2.7 主要操作流程

**1. 新建会话** 在侧边栏创建或选择一个会话。

**2. 导入视频** 选择一个或多个受支持的视频文件, 确认文件名和路径正确。

**3. 开始分析** 提交视频后等待索引状态从 processing 变为 completed。处理时间取决于视频长度、硬件和模型服务速度。

**4. 视频问答** 分析完成后输入问题, 等待回答显示在当前会话中。

**5. 保存与退出** 会话会保存到存储目录。退出后不要移动或重命名已导入的视频, 否则再次访问时可能找不到源文件。

## 2.8 停止程序与常见问题

先关闭 Electron 窗口, 再分别在两个 PowerShell 窗口按 Ctrl+C 结束桌面开发进程和 Python 后端。

| 现象             | 检查方法                                                                                     |
|----------------|------------------------------------------------------------------------------------------|
| 桌面端一直等待后端      | 确认 <code>python videorag_api.py</code> 正在运行，并检查 64451 至 64470 端口是否被防火墙阻止                 |
| 提示缺少配置         | 确认存储目录、OpenAI API Key、DashScope API Key 均已保存                                             |
| ImageBind 加载失败 | 确认 <code>imagebind_huge.pth</code> 位于存储目录下的 <code>imagebind_huge</code> 文件夹，且磁盘空间和文件权限正常 |
| 视频处理失败         | 确认 FFmpeg 可用、视频路径未变化，并查看 Python 后端终端中的错误信息                                               |
| pnpm 命令不可用     | 确认 Node.js $\geq 20$ ，并通过 Corepack 或 npm 全局安装 pnpm 9.10.0                                |

macOS 的启动顺序相同，路径改用 `/`。PyTorch 和系统依赖应按 Mac 硬件重新选择，不能直接照搬 Windows GPU 安装方式。

### 3 程序运行效果

程序运行成功后，可通过以下现象确认完整链路可用：

| 阶段    | 预期效果                                                |
|-------|-----------------------------------------------------|
| 后端启动  | Python 终端显示 Flask 服务端口，访问 /api/health 可得到 status=ok |
| 桌面端启动 | 弹出 Vimo Desktop 窗口；首次运行显示初始化向导，完成后进入主界面             |
| 模型准备  | ImageBind 下载进度达到 100%，服务状态显示可进行视频处理                 |
| 视频导入  | 界面显示所选视频的文件名、路径和大小，重复选择不会产生重复项                      |
| 视频分析  | 界面显示 processing 状态和当前步骤，完成后显示已索引视频                  |
| 视频问答  | 用户问题出现在会话中，查询完成后显示模型生成的回答                           |
| 会话恢复  | 关闭并重新启动应用后，已保存的会话标题、消息和视频信息仍可读取                     |